Flow Control Statements in Python

Flow Control Statements in Python

(Control Structures in Python)

Index - Purpose of flow control statements - Types of flow control statements 1. Conditional / Selection / Branching statements 1. Simple `if` statement 2. `if..else` statement 3. `if..elif..else` statement 4. `match case` statement 2. Looping / Iterative / Repetitive statements 1. `while` loop / `while..else` loop 2. `for` loop / `for..else` loop 3. Transfer flow statements 1. `break` 2. `continue` 3. `pass` 4. `return`

Purpose of Flow Control Statements

The purpose of flow control statements in Python is to perform a certain operation only once (in the case of **True**, perform the *x-operation*, and in the case of **False**, perform the *y-operation*), or to perform a certain operation repeatedly for a finite number of times until the test condition becomes **False**.

In Python programming, flow control statements are classified into **3 types**:

1. Conditional or Selection or Branching statements
2. Looping or Iterative or Repetitive statements
3. Transfer flow statements

1. Conditional or Selection or Branching Statements

The purpose of conditional/selection/branching statements is: > “To perform the x-operation in the case of True, or perform the y-operation in the case of > False, only once.”

In Python, we have **4 types** of conditional/selection/branching statements: 1. Simple `if` statement 2. `if..else` statement 3. `if..elif..else` statement 4. `match case` statement

1.1 Simple `if` Statement

Syntax:

```
if (Test_Cond):
    statement-1
    statement-2
    ...
    statement-n
other statement in program
```

Explanation: - Here `if` is a keyword. - The test condition can be either `True` or `False`. - If the test condition is `True`, the PVM executes the indentation block of statements, and later executes the other statements. - If the test condition is `False`, the PVM executes the other statements *without* executing the indentation block of statements.

Flowchart logic:

```
    entry
      |
[ Test Cond? ]
 False   True
   |       |
   |   Execute indentation
   |     block of statements
   |       |
   +---> Other statement in program
```

Program: Movie ticket check

```
tkt = input("Do you have a Ticket (yes/no): ")

if tkt.lower() == "yes":
    print("Watch the movie")
    print("Enjoy")

print("Go home and read python notes")
```

Debugging note: if the user's input could be "YES", "Yes", "yes" etc., comparing directly with `== "yes"` gives a wrong logic result for other cases. The correct way is to normalise the input first with `.lower()` (or `.upper()`) before comparing, as done above.

Program: Find the biggest of two numbers and check for equality (using simple `if`)

```
a = float(input("Enter any number: "))
b = float(input("Enter value of b: "))

if a > b:
    print("Big({},{})={}".format(a, b, a))

if b > a:
    print("Big({},{})={}".format(a, b, b))

if a == b:
    print("Both values are equal")
```

Program: Decide +ve, -ve or zero using simple `if` statement

```
# n = 100, -100, 0 (test cases)
n = float(input("Enter any number: "))

if n == 0:
    print("{} is zero".format(n))

if n > 0:
```

```
        print("{} is +ve".format(n))

    if n < 0:
        print("{} is -ve".format(n))
```

Program: Accept any word and decide if it is a vowel word or not

```
    word = input("Enter any word: ")  # e.g. apple, sky, dry

    if ("a" in word or "e" in word or "i" in word or "o" in word or "u" in word):
        print("{} is vowel word".format(word))

    if not ("a" in word or "e" in word or "i" in word or "o" in word or "u" in word):
        print("{} is not vowel word".format(word))
```

Program: Accept any value and decide digit / alphabet / alphanumeric / special symbol

```
    val = input("Enter any value: ")

    if val.isalpha():
        print("{} is alphabet".format(val))

    if val.isdigit():
        print("{} is digit".format(val))

    if val.isalnum() and not val.isalpha() and not val.isdigit():
        print("{} is alphanumeric".format(val))

    if not val.isalnum():
        print("{} is special symbol".format(val))
```

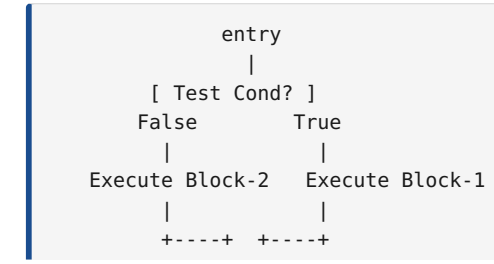
1.2 `if..else` Statement

Syntax:

```
    if (Test_Cond):
        statement-1
        statement-2
        ...
        statement-1n
    else:
        statement-11
        statement-12
        ...
        statement-1n
    other statements in program
```

Explanation: - Here `if` and `else` are keywords. - If the test condition is `True`, the PVM executes indentation Block-1 and later executes the other statements in the program. - If the test condition is `False`, the PVM executes indentation Block-2 (the `else` block) and later executes the other statements in the program.

Flowchart logic:



```
          |  |
    Other statement in program
```

Program: Accept 2 integers, find the biggest number among them (`if..else`)

```
a = float(input("Enter value of a: "))
b = float(input("Enter value of b: "))

if a > b:
    print("{} is biggest".format(a))
else:
    if b > a:
        print("{} is biggest".format(b))
    else:
        print("Both values are equal")
```

Program: Accept any word and decide whether it is a palindrome or not

```
a = input("Enter a word: ")

if a.lower() == a[::-1].lower():
    print("Palindrome")
else:
    print("Not Palindrome")
```

Program: Accept any value, decide digit / alpha / alphanumeric / special symbol (`if..else`)

```
val = input("Enter any value: ")

if val.isalpha():
    print("{} is alphabet".format(val))
else:
    if val.isdigit():
        print("{} is digit".format(val))
    else:
        if val.isalnum():
            print("{} is alnum".format(val))
        else:
            print("{} is special symbol".format(val))
```

Program: Accept any digit and display its name (nested `if..else`)

This is the version done with *nested if..else* blocks. Also handles multi-digit +ve / -ve numbers that are not single digits 0-9.

```
d = int(input("Enter value: "))

if d == 0:
    print("{} is zero".format(d))
else:
    if d == 1:
        print("{} is one".format(d))
    else:
        if d == 2:
            print("{} is two".format(d))
        else:
            if d == 3:
                print("{} is three".format(d))
            else:
                if d == 4:
                    print("{} is four".format(d))
```

```
                    else:
                        if d == 5:
                            print("{} is five".format(d))
                        else:
                            if d == 6:
                                print("six")
                            else:
                                if d == 7:
                                    print("seven")
                                else:
                                    if d == 8:
                                        print("eight")
                                    else:
                                        if d == 9:
                                            print("nine")
                                        else:
                                            if d not in [0,1,2,3,4,5,6,7,8,9] and d > 0:
                                                print("{} is +ve no.".format(d))
                                            else:
                                                print("{} is -ve no.".format(d))
```

1.3 `if..elif..else` Statement

Syntax:

```
if (Test_Cond):
    Block of stmts-1
elif (Test_Cond2):
    Block of stmts-2
elif (Test_Cond3):
    Block of stmts-3
elif (Test_Cond-n):
    Block of stmts-n
else:
    else block of stmts
other stmts in program
```

Explanation: - `elif` is a keyword. - If Test_Cond-1 is `True`, PVM executes Block of stmts-1 and other statements. - If Test_Cond-1 is `False` and Test_Cond-2 is `True`, PVM executes Block of stmts-2 and other statements. - This process repeats until all test conditions are evaluated. If all test conditions are `False`, PVM executes the `else` block of statements and other statements in program. - Writing the `else` block is **optional**.

Program: Rewrite the digit-name program using `if..elif..else`

```
d = int(input("Enter any digit: "))

if d == 0:
    print("Zero")
elif d == 1:
    print("One")
elif d == 2:
    print("Two")
elif d == 3:
    print("Three")
elif d == 4:
    print("Four")
elif d == 5:
    print("Five")
elif d == 6:
    print("Six")
elif d == 7:
    print("Seven")
elif d == 8:
```

```
        print("Eight")
    elif d == 9:
        print("Nine")
    elif d not in [0,1,2,3,4,5,6,7,8,9] and d > 0:
        print("{} is +ve no.".format(d))
    else:
        print("{} is -ve no.".format(d))
```

Program: Same digit-name lookup using a dictionary

```
dodj = {0: "zero", 1: "one", 2: "two", 3: "three", 4: "four",
        5: "five", 6: "six", 7: "seven", 8: "eight", 9: "nine"}

d = int(input("Enter any digit: "))
res = dodj.get(d) if dodj.get(d) is not None else ("+ve no" if d > 0 else "-ve no")

print("{} is {}".format(d, res))
```

1.4 `match case` Statement

`match case` is one of the new features introduced in Python **3.10** onwards.

The purpose of the `match case` statement is: > “To deal with pre-designed conditions or menu driven applications.”

Syntax:

```
match (choice_expr):
    case choice_label_1:
        Block of st-1
    case choice_label_2:
        Block of st-2
    ...
    case choice_label_n:
        Block of st-n
    case _:                 # Default case block
        default Block of statement
other statements in program
```

Explanation: - `match` and `case` are keywords. - `choice_expr` represents either `int`, `str` or `bool`. - If `choice_expr` matches `case_label_1`, PVM executes Block of statements-1 and later executes other statements in program. Similarly for `case_label_2` … `case_label_n`. - If `choice_expr` does not match any of the specified case labels, PVM executes the default block of statements written under the default case block (`case _:`) and later executes other statements in program. - Writing the default case block is **optional**, and if written it must be written **last** (otherwise a `SyntaxError` occurs). - When multiple case labels are grouped under one `case`, they must be combined with the bitwise OR operator `|`.

Program: Menu-driven application - Arithmetic Operations (using `match case`)

```
print("Arithmetic Operations")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Division")
print("5. Modulo Division")
print("6. Exponentiation")
print("7. Exit")

ch = int(input("Enter your choice: "))
```

```
match ch:
    case 1:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Sum =", a + b)
    case 2:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Difference =", a - b)
    case 3:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Product =", a * b)
    case 4:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Quotient =", a / b)
    case 5:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Remainder =", a % b)
    case 6:
        a, b = map(float, input("Enter two values separated by space: ").split())
        print("Power =", a ** b)
    case 7:
        print("Exiting...")
    case _:
        print("Invalid choice")
```

Program: Menu-driven application - Area of Different Figures (using `match case`)

```
print("R. Rectangle")
print("S. Square")
print("C. Circle")
print("T. Triangle")
print("E. Exit")

ch = input("Enter your choice: ").upper()

match ch:
    case "R":
        l, w = map(float, input("Enter length and width separated by space: ").split())
        print("Area of Rectangle =", l * w)
    case "S":
        s = float(input("Enter side: "))
        print("Area of Square =", s * s)
    case "C":
        r = float(input("Enter radius: "))
        print("Area of Circle =", 3.14159 * r * r)
    case "T":
        b, h = map(float, input("Enter base and height separated by space: ").split())
        print("Area of Triangle =", 0.5 * b * h)
    case "E":
        print("Exiting...")
    case _:
        print("Invalid choice")
```

Program: Temperature Conversion Calculator (using `match case`)

```
print("1. Fahrenheit to Celsius")
print("2. Fahrenheit to Kelvin")
print("3. Celsius to Fahrenheit")
print("4. Celsius to Kelvin")
print("5. Kelvin to Fahrenheit")
print("6. Kelvin to Celsius")
print("7. Exit")

ch = int(input("Enter your choice: "))

match ch:
    case 1:
```

```
            f = float(input("Enter Fahrenheit: "))
            print("Celsius =", (f - 32) * 5 / 9)
        case 2:
            f = float(input("Enter Fahrenheit: "))
            print("Kelvin =", (f - 32) * 5 / 9 + 273.15)
        case 3:
            c = float(input("Enter Celsius: "))
            print("Fahrenheit =", c * 9 / 5 + 32)
        case 4:
            c = float(input("Enter Celsius: "))
            print("Kelvin =", c + 273.15)
        case 5:
            k = float(input("Enter Kelvin: "))
            print("Fahrenheit =", (k - 273.15) * 9 / 5 + 32)
        case 6:
            k = float(input("Enter Kelvin: "))
            print("Celsius =", k - 273.15)
        case 7:
            print("Exiting...")
        case _:
            print("Invalid choice")
```

Notes

- The remaining index items (Looping / Iterative statements, Transfer flow statements, and combined programs) were listed in the index of the original notes but not yet written out in the pages photographed — this notebook covers everything on the pages provided (Conditional / Selection / Branching statements: simple `if`, `if..else`, `if..elif..else`, and `match case`, with all worked programs completed and made runnable).